

Journée Julia pour
les statistiques et science des données
du Groupe Calcul CNRS

Optimiser en Julia avec JuMP

Xavier Gandibleux
Nantes Université, France

15 Juin 2026

Pourquoi cette aventure avec Julia et JuMP?

Travaux en optimisation multi-objectif

$$\min \left\{ Fx : x \in X \right\}$$

2015, kick-off du projet de recherche DFG/ANR **vOpt**

Sous-tâche 2.2: Analyse expérimentale et développement d'un prototype.

- ▶ Programmation mathématique multi-objectif en variables mixtes

$$\min \left\{ Cx + Dy : Ax + By = d, x \geq 0, y \in \mathbb{Z} \right\}$$

- ▶ Consortium réunissant optimiseurs (mathématiciens et informaticiens)
- ▶ Choix naturel du C ou C++ (informaticiens)
- ▶ Pratique de Matlab (mathématiciens)

Pourquoi cette aventure avec Julia et JuMP?

Travaux en optimisation multi-objectif

$$\min \left\{ Fx : x \in X \right\}$$

2015, kick-off du projet de recherche DFG/ANR **vOpt**

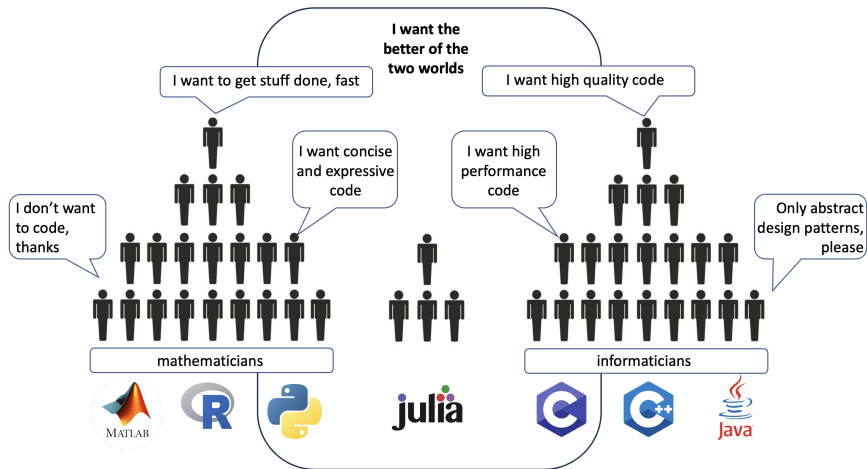
Sous-tâche 2.2: Analyse expérimentale et développement d'un prototype.

- ▶ Programmation mathématique multi-objectif en variables mixtes

$$\min \left\{ Cx + Dy : Ax + By = d, x \geq 0, y \in \mathbb{Z} \right\}$$

- ▶ Consortium réunissant optimiseurs (mathématiciens et informaticiens)
- ▶ Choix naturel du C ou C++ (informaticiens)
- ▶ Pratique de Matlab (mathématiciens)

Facile, haut-niveau, efficace, expressif $\Rightarrow$ Julia



Syntaxe....

C

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int A[100][100][100];
    int i,j,k,loop;
    for (loop=0; loop <1000; loop++){
        for (i=0; i<100; i++){
            for (j=0; j<100; j++){
                for (k=0; k<100; k++){
                    A[i][j][k] = loop;
                }
            }
        }
        printf("%d\n", A[99][99][99]);
    }
    return 0;
}
```

Julia

```
@time begin

    A = zeros(100,100,100)
    for loop = 1:1000
        for i = 1:100
            for j = 1:100
                for k = 1:100
                    A[i,j,k] = loop
                end
            end
        end
        println(A[100,100,100])
    end

end
```

MATLAB

```
tic

A = zeros(100,100,100);
for loop = 1:1000
    for i = 1:100
        for j = 1:100
            for k = 1:100
                A(i,j,k) = loop;
            end
        end
    end
    disp(A(100,100,100));
end

toc
```

.... ok!

Rapidité....

► C

without optimization option

1.286582 seconds

with optimization option -O3

0.087682 seconds

► Julia

1-to-1 straightforward implementation

26.732819 seconds (489.05 M allocations: 7.297 GiB, 0.29% gc time)

julia style compliant implementation

0.252995 seconds (22.39 k allocations: 8.499 MiB)

polished implementation

0.114872 seconds (20.75 k allocations: 4.318 MiB)

.... ok!

Objectif pour aujourd'hui

Coder et résoudre un programme mathématique

Soit...

$$\left[\begin{array}{llll} \max z(x) = & x_1 & + & 3x_2 \\ \text{s.t} & x_1 & + & x_2 \leq 14 & (1) \\ & -2x_1 & + & 3x_2 \leq 12 & (2) \\ & 2x_1 & - & x_2 \leq 12 & (3) \\ & x_1 & , & x_2 \geq 0 & (4) \end{array} \right]$$

...à solutionner

D'un (MI)LP à la recommandation d'une solution

- JuMP
- GNU MP
- Pyomo...

- AMPL
- GAMS
- ZIMPL...

modéliser

- MPS
- LP...

formulation bas-niveau
(flat form)

résoudre

- HiGHS
- GLPK
- SCIP...

modèle de haut-niveau
(structured form)

- GUROBI
- CPLEX
- Xpress...

analyser

- ...

Un **langage de modélisation algébrique** en Julia ...



... pour l'**optimisation mathématique** (linéaire, discrète, mixte, etc.)

Regard sur JuMP

- ▶ **Convivial**: une syntaxe qui imite les expressions d'un programme mathématique.
- ▶ **Rapide**: vitesses comparables à celles des langages de modélisation tels que AMPL.
- ▶ **Indépendance avec le solveur**: actuellement, 59 solveurs sont supportés

Parmi les solveurs (M)LP : GLPK, Cbc, Clp, HiGHS, SCIP, CPLEX, Gurobi, FICO Xpress, COPT...
- ▶ **Facilité d'intégration**: JuMP est entièrement écrit en Julia. Les solveurs constituent les seules dépendances binaires.

Regard sur JuMP

- ▶ **Convivial**: une syntaxe qui imite les expressions d'un programme mathématique.
- ▶ **Rapide**: vitesses comparables à celles des langages de modélisation tels que AMPL.
- ▶ **Indépendance avec le solveur**: actuellement, 59 solveurs sont supportés

Parmi les solveurs (M)LP : GLPK, Cbc, Clp, HiGHS, SCIP, CPLEX, Gurobi, FICO Xpress, COPT...
- ▶ **Facilité d'intégration**: JuMP est entièrement écrit en Julia. Les solveurs constituent les seules dépendances binaires.

Regard sur JuMP

- ▶ **Convivial**: une syntaxe qui imite les expressions d'un programme mathématique.
- ▶ **Rapide**: vitesses comparables à celles des langages de modélisation tels que AMPL.
- ▶ **Indépendance avec le solveur**: actuellement, 59 solveurs sont supportés

Parmi les solveurs (MI)LP : GLPK, Cbc, Clp, HiGHS, SCIP, CPLEX, Gurobi, FICO Xpress, COPT...

- ▶ **Facilité d'intégration**: JuMP est entièrement écrit en Julia. Les solveurs constituent les seules dépendances binaires.

Regard sur JuMP

- ▶ **Convivial**: une syntaxe qui imite les expressions d'un programme mathématique.
- ▶ **Rapide**: vitesses comparables à celles des langages de modélisation tels que AMPL.
- ▶ **Indépendance avec le solveur**: actuellement, 59 solveurs sont supportés

Parmi les solveurs (M)LP : GLPK, Cbc, Clp, HiGHS, SCIP, CPLEX, Gurobi, FICO Xpress, COPT...

- ▶ **Facilité d'intégration**: JuMP est entièrement écrit en Julia. Les solveurs constituent les seules dépendances binaires.

Récupérer le notebook jupyter

`RRNantes2026.ipynb`

à l'adresse :

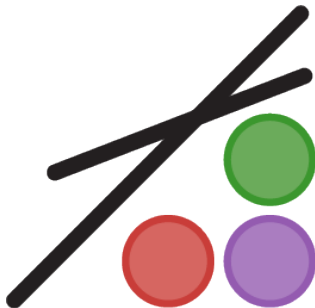
`https://github.com/xgandibleux/RR2026`

dans le dossier

`notebooks`

JuMP (partie 1)

Modélisation explicite

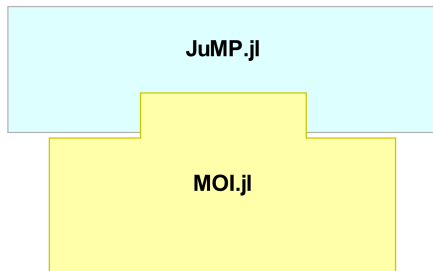


version 1.30.0 et plus

Ajout des packages à la distribution Julia

A faire une seule fois

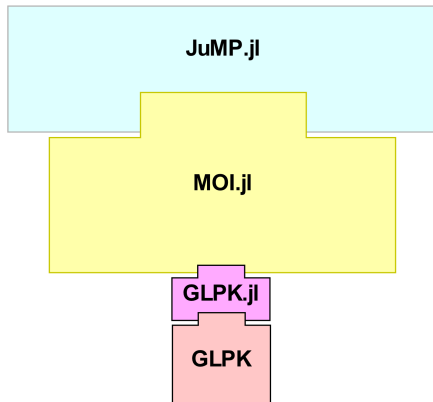
```
julia> using Pkg  
julia> Pkg.add("JuMP")
```



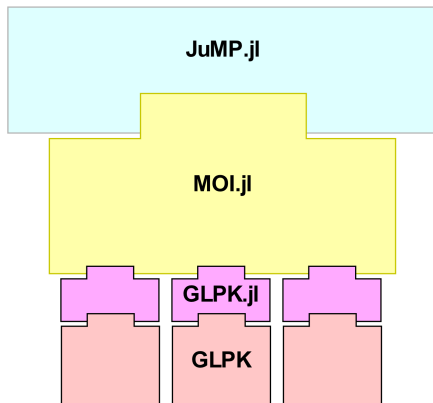
JuMP utilise *MathOptInterface (MOI)*, une couche d'abstraction conçue pour offrir une interface unifiée aux solveurs d'optimisation mathématique.

A faire une seule fois

```
julia> Pkg.add("GLPK")
```



A faire une seule fois



Modélisation et résolution d'un problème (MI)LP avec JuMP

A faire à chaque session

```
julia> using JuMP, GLPK
```

Codage du problème d'optimisation (1/4)

Créer un modèle :

```
model = Model(solver)
```

```
julia> modLP = Model(GLPK.Optimizer)
```

Résultat:

[]

Codage du problème d'optimisation (2/4)

Définir une variable :

```
@variable(model, variable)
```

```
julia> @variable(modLP, x1 >= 0)
```

```
julia> @variable(modLP, x2 >= 0)
```

Résultat :

$$\left[\begin{array}{ccc} x_1 & , & x_2 \end{array} \geq 0 \quad (4) \right]$$

Codage du problème d'optimisation (3/4)

Définition d'une fonction objectif :

```
@objective(model, sense, expression)
```

```
julia> @objective(modLP, Max, x1 + 3x2)
```

Résultat :

$$\left[\begin{array}{rcll} \max z(x) = & x_1 & + & 3x_2 & (0) \\ & x_1 & , & x_2 & \geq 0 & (4) \end{array} \right]$$

Codage du problème d'optimisation (4/4)

Définition d'une contrainte :

```
@constraint(model, id, constraint)
```

```
julia> @constraint(modLP, cst1, x1 + x2 <= 14)
julia> @constraint(modLP, cst2, -2x1 + 3x2 <= 12)
julia> @constraint(modLP, cst3, 2x1 - x2 <= 12)
```

Résultat :

$$\left[\begin{array}{rcllcl} \max z(x) = & x_1 & + & 3x_2 & & (0) \\ s.t & x_1 & + & x_2 & \leq & 14 & (1) \\ & -2x_1 & + & 3x_2 & \leq & 12 & (2) \\ & 2x_1 & - & x_2 & \leq & 12 & (3) \\ & x_1 & , & x_2 & \geq & 0 & (4) \end{array} \right]$$

Résolution du problème d'optimisation

```
optimize!(model)
```

```
julia> optimize!(modLP)
```

Extraire le résultat

```
julia> @show termination_status(modLP)
```

```
julia> @show objective_value(modLP)
```

```
julia> @show value(x1)
```

```
julia> @show value(x2)
```

Extraire le résultat

```
julia> @show termination_status(modLP)
```

```
julia> @show objective_value(modLP)
```

```
julia> @show value(x1)
```

```
julia> @show value(x2)
```

Extraire le résultat

```
julia> @show termination_status(modLP)
```

```
julia> @show objective_value(modLP)
```

```
julia> @show value(x1)
```

```
julia> @show value(x2)
```

Concernant les variables

- ▶ Par défaut, les variables sont **continues** et **non bornées** :

```
julia> @variable(model, x)           # x est libre
```

- ▶ spécifier des limites inférieures et/ou supérieures d'une variable :

```
julia> @variable(model, x≥lb))      # x est bornée  
julia> @variable(model, x≤ub)  
julia> @variable(model, lb≤x≤ub)  
julia> @variable(model, x==2)       # x est fixée
```

- ▶ spécifier le type d'une variable discrète :

```
julia> @variable(model, x≥0, Int)    #  $x \in \mathbb{N}$   
julia> @variable(model, x, Bin)      #  $x \in \{0, 1\}$ 
```

Concernant les variables

- ▶ Par défaut, les variables sont **continues** et **non bornées** :

```
julia> @variable(model, x)           # x est libre
```

- ▶ spécifier des limites inférieures et/ou supérieures d'une variable :

```
julia> @variable(model, x ≥ lb))    # x est bornée  
julia> @variable(model, x ≤ ub)  
julia> @variable(model, lb ≤ x ≤ ub)  
julia> @variable(model, x == 2)     # x est fixée
```

- ▶ spécifier le type d'une variable discrète :

```
julia> @variable(model, x ≥ 0, Int)  #  $x \in \mathbb{N}$   
julia> @variable(model, x, Bin)     #  $x \in \{0, 1\}$ 
```

Concernant les variables

- ▶ Par défaut, les variables sont **continues** et **non bornées** :

```
julia> @variable(model, x)           # x est libre
```

- ▶ spécifier des limites inférieures et/ou supérieures d'une variable :

```
julia> @variable(model, x≥lb))      # x est bornée  
julia> @variable(model, x≤ub)  
julia> @variable(model, lb≤x≤ub)  
julia> @variable(model, x==2)       # x est fixée
```

- ▶ spécifier le type d'une variable discrète :

```
julia> @variable(model, x≥0, Int)    #  $x \in \mathbb{N}$   
julia> @variable(model, x, Bin)     #  $x \in \{0, 1\}$ 
```

Résumé

```
julia> using JuMP, GLPK

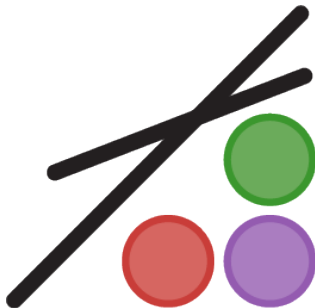
julia> modLP = Model(GLPK.Optimizer)
julia> @variable(modLP, x1 >= 0)
julia> @variable(modLP, x2 >= 0)
julia> @objective(modLP, Max, x1 + 3x2)
julia> @constraint(modLP, cst1, x1 + x2 <= 14)
julia> @constraint(modLP, cst2, -2x1 + 3x2 <= 12)
julia> @constraint(modLP, cst3, 2x1 - x2 <= 12)

julia> optimize!(modLP)

julia> @show termination_status(modLP)
julia> @show objective_value(modLP)
julia> @show value(x1), value(x2)
```

JuMP (partie 2)

Modélisation implicite



version 1.30.0 et plus

Ecrire un modèle implicite (1/5)

Exemple (suite):

$$\left[\begin{array}{llll} \max f(x) = & x_1 & + & 3x_2 & (0) \\ \text{s.t} & x_1 & + & x_2 & \leq 14 \quad (1) \\ & -2x_1 & + & 3x_2 & \leq 12 \quad (2) \\ & 2x_1 & - & x_2 & \leq 12 \quad (3) \\ & x_1 & , & x_2 & \geq 0 \quad (4) \end{array} \right]$$

↓

$$c = (1, 3), \quad d = (14, 12, 12), \quad T = \begin{pmatrix} 1 & 1 \\ -2 & 3 \\ 2 & -1 \end{pmatrix}$$

$$\left[\begin{array}{llll} \max f(x) = & \sum_{j=1}^2 c_j x_j & & (0) \\ \text{s.t} & \sum_{j=1}^2 t_{ij} x_j \leq d_i & i = 1, 3 & (1 - 3) \\ & x_j \geq 0 & j = 1, 2 & (4) \end{array} \right]$$

Ecrire un modèle implicite (2/5)

Exemple (suite):

$$c = (1, 3), \quad d = (14, 12, 12), \quad T = \begin{pmatrix} 1 & 1 \\ -2 & 3 \\ 2 & -1 \end{pmatrix}$$

```
julia> c = [1, 3]
julia> d = [14, 12, 12]
julia> T = [1 1 ; -2 3; 2 -1]
julia> n,m = size(T)
```

Ecrire un modèle implicite (2/5)

Exemple (suite):

$$\left[\begin{array}{lll} \max z(x) = & \sum_{j=1}^2 c_j x_j & (0) \\ \text{s.t} & \sum_{j=1}^2 t_{ij} x_j \leq d_i & i = 1, 3 \quad (1-3) \\ & x_j \geq 0 & j = 1, 2 \quad (4) \end{array} \right]$$

```
julia> using JuMP, GLPK
julia> md = Model(GLPK.Optimizer)
```

```
julia> @variable(md, x[1:m] ≥ 0)
```

```
julia> @objective(md, Max, sum(c[j]*x[j] for j=1:m))
```

```
julia> @constraint(md, cst[i=1:n], sum(T[i,j]*x[j]
                                     for j=1:m) ≤ d[i])
```

Ecrire un modèle implicite (2/5)

Exemple (suite):

$$\left[\begin{array}{lll} \max z(x) = & \sum_{j=1}^2 c_j x_j & (0) \\ \text{s.t} & \sum_{j=1}^2 t_{ij} x_j \leq d_i & i = 1, 3 \quad (1-3) \\ & x_j \geq 0 & j = 1, 2 \quad (4) \end{array} \right]$$

```
julia> using JuMP, GLPK
julia> md = Model(GLPK.Optimizer)
```

```
julia> @variable(md, x[1:m] ≥ 0)
```

```
julia> @objective(md, Max, sum(c[j]*x[j] for j=1:m))
```

```
julia> @constraint(md, cst[i=1:n], sum(T[i,j]*x[j]
                                     for j=1:m) ≤ d[i])
```

Ecrire un modèle implicite (2/5)

Exemple (suite):

$$\left[\begin{array}{ll} \max z(x) = & \sum_{j=1}^2 c_j x_j & (0) \\ \text{s.t} & \sum_{j=1}^2 t_{ij} x_j \leq d_i & i = 1, 3 \quad (1-3) \\ & x_j \geq 0 & j = 1, 2 \quad (4) \end{array} \right]$$

```
julia> using JuMP, GLPK
julia> md = Model(GLPK.Optimizer)
```

```
julia> @variable(md, x[1:m] ≥ 0)
```

```
julia> @objective(md, Max, sum(c[j]*x[j] for j=1:m))
```

```
julia> @constraint(md, cst[i=1:n], sum(T[i,j]*x[j]
                                     for j=1:m) ≤ d[i])
```

Ecrire un modèle implicite (2/5)

Exemple (suite):

$$\left[\begin{array}{ll} \max z(x) = & \sum_{j=1}^2 c_j x_j & (0) \\ \text{s.t} & \sum_{j=1}^2 t_{ij} x_j \leq d_i & i = 1, 3 \quad (1-3) \\ & x_j \geq 0 & j = 1, 2 \quad (4) \end{array} \right]$$

```
julia> using JuMP, GLPK
julia> md = Model(GLPK.Optimizer)
```

```
julia> @variable(md, x[1:m] ≥ 0)
```

```
julia> @objective(md, Max, sum(c[j]*x[j] for j=1:m))
```

```
julia> @constraint(md, cst[i=1:n], sum(T[i,j]*x[j]
                                     for j=1:m) ≤ d[i])
```

Variantes autour de la variable structurée

Variables structurées :

- ▶ 1-indexées...

```
julia> @variable(md, x[1:4] ≥ 0)
```

```
julia> @variable(md, x[1:2, 1:2] ≥ 0, Int)
```

- ▶ ...mais pas nécessairement...

```
julia> @variable(md, x[-4:4] ≥ 0)
```

- ▶ ...ni nécessairement numériques...

```
julia> @variable(md, x[:, :train], Bin)
```

Concernant la solution

Valeurs des solutions primales pour une variable scalaire :

```
value(variable)
```

```
julia> value(x1)
```

Valeurs des solutions primales pour une variable structurée :

```
value(variable [index])
```

```
julia> value(x[2])
```

```
value. (variable)
```

```
julia> value.(x)
```

Concernant la solution

Valeurs des solutions primales pour une variable scalaire :

```
value(variable)
```

```
julia> value(x1)
```

Valeurs des solutions primales pour une variable structurée :

```
value(variable [index])
```

```
julia> value(x[2])
```

```
value.(variable)
```

```
julia> value.(x)
```

Concernant la solution

Valeurs des solutions primales pour une variable scalaire :

```
value(variable)
```

```
julia> value(x1)
```

Valeurs des solutions primales pour une variable structurée :

```
value(variable [index])
```

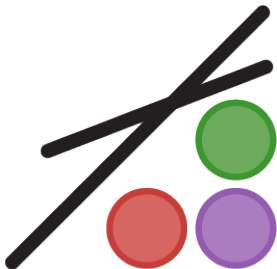
```
julia> value(x[2])
```

```
value. (variable)
```

```
julia> value. (x)
```

JuMP (partie 3) et MultiObjectiveAlgorithms

De un à plusieurs objectifs



version 1.30.0 et plus



version 1.11.0 et plus

MOP : Notations et définitions

$$\begin{array}{ll} \min & F(x) \\ \text{s.t.} & x \in X \end{array}$$

où:

$$\begin{array}{ll} X \subseteq \mathbb{R}^n & \longrightarrow \text{l'ensemble des solutions réalisables} \\ Y = F(X) \subseteq \mathbb{R}^p & \longrightarrow \text{l'ensemble des images} \end{array}$$

Objectif : calcul des ensembles Y_N et X_E pour MOP:

$$y = F(x)$$

$y^* \in Y$ est non-dominé, si $\nexists y \in Y$ tel que $y_i \leq y_i^*, \forall i$ et $y \neq y^*$.

Y_N est l'ensemble des points non-dominés.

$x^* \in X$ est efficace si y^* est non-dominé.

X_E est un ensemble complet de solutions efficaces.

A faire une seule fois

Examen de la configuration actuelle

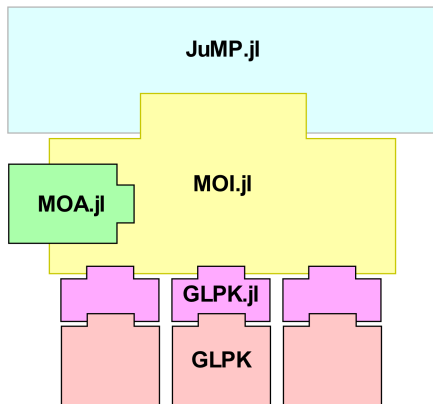
```
julia> using Pkg
```

```
julia> Pkg.status()  
Status `~/julia/environments/v1.10/Project.toml`  
 [60bf3e95] GLPK v1.2.1  
 [4076af6c] JuMP v1.23.5
```

Installation du package

```
julia> Pkg.add("MultiObjectiveAlgorithms")
```

A faire une seule fois



Parmi les algorithms actuellement disponibles (v1.11.0)

Algorithme :

- ▶ `EpsilonConstraint()`

Y.V. Haimes, L.S. Lasdon, D.A. Wismer (1971). On a bicriterion formation of the problems of integrated system identification and system optimization. *IEEE Transactions on Systems, Man and Cybernetics*, Volume SMC-1, Issue 3, Pages 296-297.

Attributs:

- ▶ `MOA.EpsilonConstraintStep()`

algorithme utilise cette valeur comme ϵ pour partitionner l'espace du premier objectif. La valeur par défaut est 1, de sorte que, pour un programme en nombres entiers purs, cet algorithme énumérera Y_N .

- ▶ `MOA.SolutionLimit()`

Si cet attribut est défini, au lieu d'utiliser le `MOA.EpsilonConstraintStep`, `EpsilonConstraint` échantillonne Y_N (division de l'intervalle du domaine du premier objectif dans l'espace des objectifs par `SolutionLimit` afin d'obtenir la valeur de ϵ à utiliser lors de l'itération). Ainsi, au plus `SolutionLimit` solutions sont retournées.

Exemple (1/5)

Calculer X_E et Y_N pour le MOP suivant :

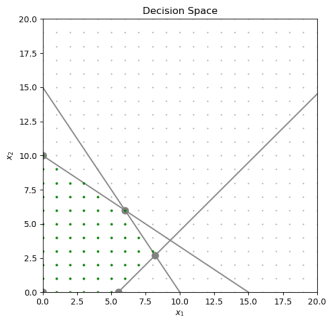
$$\begin{array}{rclclcl} \max z_1 & = & x_1 & + & x_2 & & \\ \min z_2 & = & x_1 & + & 3x_2 & & \\ s.t. & & 2x_1 & + & 3x_2 & \leq & 30 \\ & & 3x_1 & + & 2x_2 & \leq & 30 \\ & & x_1 & - & x_2 & \leq & 5.5 \\ & & x_1 & , & x_2 & \in & \mathbb{N} \end{array}$$

Exemple (2/5)

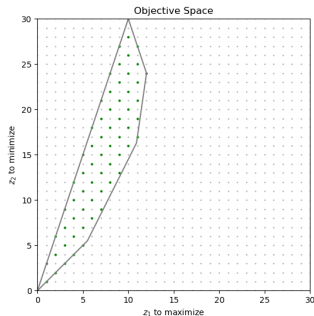
$$\begin{array}{llllll} \max z_1 & = & x_1 & + & x_2 & \\ \min z_2 & = & x_1 & + & 3x_2 & \\ \text{s.t.} & & 2x_1 & + & 3x_2 & \leq 30 \\ & & 3x_1 & + & 2x_2 & \leq 30 \\ & & x_1 & - & x_2 & \leq 5.5 \\ & & x_1 & , & x_2 & \in \mathbb{N} \end{array}$$

```
julia> c1 = [1,1]
julia> c2 = [1,3]
julia> A = [2 3; 3 2; 1 -1]
julia> b = [30, 30, 5.5]
```

Espace des décisions :



Espace des objectifs :



Exemple (3/5)

Codage du problème MOO avec JuMP

```
julia> using JuMP, GLPK
julia> import MultiObjectiveAlgorithms as MOA
```

```
julia> model = Model( )
julia> @variable(model, x1 ≥ 0, Int)
julia> @variable(model, x2 ≥ 0, Int)
julia> @expression(model, fct1, x1 + x2)           # à maximiser
julia> @expression(model, fct2, x1 + 3 * x2)       # à minimiser
julia> @objective(model, Max, [fct1, (-1) * fct2])
julia> @constraint(model, 2*x1 + 3*x2 ≤ 30)
julia> @constraint(model, 3*x1 + 2*x2 ≤ 30)
julia> @constraint(model, x1 - x2 ≤ 5.5)
```

Exemple (4/5)

Sélection du solveur MIP : e.g. GLPK

```
julia> set_optimizer(model, ()->MOA.Optimizer(GLPK.Optimizer))
```

Sélection de l'algorithme : ϵ -constraint; step=1 (valeur par défaut)

```
julia> set_attribute(model, MOA.Algorithm(), MOA.EpsilonConstraint())
```

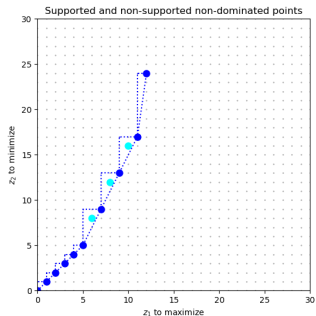
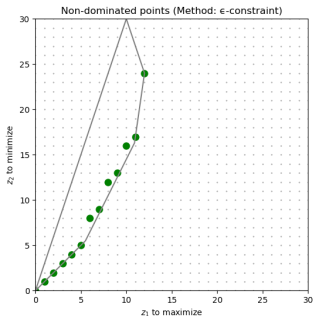
Optimisation du problème MOO :

```
julia> optimize!(model)
```

Exemple (5/5)

Obtention de X_E et Y_N

```
julia> for i in 1:result_count(model)
julia>     z1_opt = objective_value(model; result = i)[1]
julia>     z2_opt = -1 * objective_value(model; result = i)[2]
julia>     x1_opt = value(x1; result = i)
julia>     x2_opt = value(x2; result = i)
julia> end
```



Affichage graphique de Y_N

```
using Plots
using LaTeXStrings

Z1_opt = []; Z2_opt = []
for i in 1:result_count(model)
    push!(Z1_opt, objective_value(model; result = i)[1])
    push!(Z2_opt, -1 * objective_value(model; result = i)[2])
end

plot(Z1_opt, Z2_opt,
     title = "Points non-dominés (méthode e-constraint)",
     xlabel = L"$z_1$ to maximize",
     xlabel = L"$z_2$ to minimize",
     xlims = (0, 30),
     ylims = (0, 30),
     seriestype = :scatter,
     marker      = :circle,
     markersize  = 5,
     color       = :green,
     legend      = false
)
```

En guise de conclusion

C'est loin d'être fini...

Aller plus loin avec JuMP :

- ▶ modifier les données du modèle
- ▶ modifier le modèle lui-même
- ▶ usages avancés en modélisation dont ensembles et indices
- ▶ usages avancés en résolution dont callback vers les solveurs
- ▶ autres classes de problèmes d'optimisation

Références (1/2)...

JuMP:

<https://jump.dev/> et <https://jump.dev/JuMP.jl/stable/>

Miles Lubin, Oscar Dowson, Joaquim Dias Garcia, Joey Huchette, Benoît Legat, Juan Pablo Vielma. JuMP 1.0: Recent improvements to a modeling language for mathematical optimization. *Mathematical Programming Computation*. 2023.

MultiObjectiveAlgorithms:

<https://github.com/jump-dev/MultiObjectiveAlgorithms.jl>

Oscar Dowson, Xavier Gandibleux, Gökhan Kof. MultiObjectiveAlgorithms.jl: a Julia package for solving multi-objective optimization problems. *INFORMS Journal of Computing*, 2026 (to appear).

Xavier Gandibleux. Native Multi-Objective Mathematical Programming Software. Chapter 15 in *Handbook on Multi-objective Combinatorial Optimisation* (M. Ehrgott, J. Figueira, Ö. Karsu, L. Paquete Edts.). Springer, 2026 (to appear).

Références (2/2)...

Livres

Julia Programming for Operations Research

Changhyun Kwon

2nd Edition. 260 pages. ISBN-10: 1798205475, ISBN-13: 9781798205471.

<https://www.chkwon.net/julia/juliabook/juliabook2.html>

Algorithms for Optimization

Mykel J. Kochenderfer, and Tim A. Wheeler

The MIT Press Cambridge, Massachusetts London, England

<https://algorithmsbook.com/optimization/files/optimization.pdf>

Statistics with Julia:

Fundamentals for Data Science, Machine Learning and Artificial Intelligence

Yoni Nazarathy and Hayden Klok

2021, Springer

<https://statisticswithjulia.org/>

Cours

Practical Scientific Computing

Urbain Vaes

École des Ponts (Paris)

<https://enpc-casc.github.io/docs/index.html>

Exercice

Analyse discriminante et programmation linéaire

A linear programming approach to the discriminant problem.

Ned Freed and Fred Glover.

Decision Sciences, volume 12, issue 1, pages 68-74. 1981

Enoncé

But :

Étant donné des observations appartenant à deux groupes **connus**, trouver une règle de classification permettant d'affecter une **nouvelle observation** à l'un des deux groupes.

Exemple (Freed & Glover, 1981) :

- ▶ 10 employés, 2 caractéristiques (x_1 = expérience, x_2 = formation)

- ▶ **Groupe Succès**

j	x_1	x_2
1	4	1
2	5	2
3	7	2
4	9	1
5	8	4

- Groupe Échec**

j	x_1	x_2
6	1	1
7	3	1
8	2	2
9	6	3
10	3	3

Approche pour 2 variables explicatives

Chercher **hyperplan séparateur** :

- ▶ un score :

$$\text{score}(j) = x_{1,j} \times w_1 + x_{2,j} \times w_2$$

- ▶ un seuil c tel que :

- ▶ $\text{score}(j) \geq c$ pour tout individu du groupe *Succès*
- ▶ $\text{score}(j) \leq c$ pour tout individu du groupe *Échec*

Critère de qualité : on souhaite que les scores soient loin du seuil c .

- ▶ une marge d :

- ▶ Succès $\Rightarrow \text{score}(j) \geq c + d$
- ▶ Échec $\Rightarrow \text{score}(j) \leq c - d$

Plus d est grand, plus les groupes sont **bien séparés**.

Généralisation à p variables explicatives :

- ▶ $p + 1$ variables de décision $(w_1, \dots, w_p, d)$

Modèle LP (pour 2 variables explicatives)

Soit

- ▶ n_S observations dans le groupe *Succès*
- ▶ n_E observations dans le groupe *Échec*

posons

$$\left[\begin{array}{ll} \max & d \\ \text{s.t} & x_{1,j}^s \times w_1 + x_{2,j}^s \times w_2 - d \geq c \quad \forall j \in \{1, \dots, n_S\} \\ & x_{1,j}^e \times w_1 + x_{2,j}^e \times w_2 + d \leq c \quad \forall j \in \{1, \dots, n_E\} \\ & w_1, w_2, d \in \mathbb{R} \end{array} \right]$$

avec comme variables de décision :

- ▶ w_1, w_2 : poids (non contraints en signe)
- ▶ d : marge (non contrainte en signe)

et comme une valeur donnée :

- ▶ c : seuil (non négatif)

Résultats

Données :

```
X_s = [ 4 1 ;      # une ligne = une observation
        5 2 ;
        7 2 ;
        9 1 ;
        8 4 ]
```

```
X_e = [ 1 1 ;
        3 1 ;
        2 2 ;
        6 3 ;
        3 3 ]
```

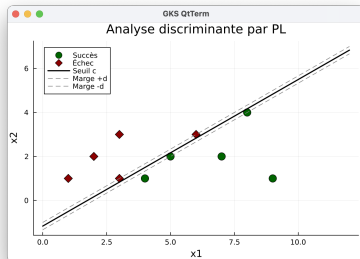
```
c = 10
```

Solution :

```
w1 = 5.71   w2 = -8.57   d = 1.43
```

```
Scores Succès : [14.3, 11.4, 22.9, 42.9, 11.4]
```

```
Scores Échec  : [-2.9, 8.6, -5.7, 8.6, -8.6]
```



Réponse

```
# ----- DONNEES -----  
  
X_s = [ 4 1 ;    # une ligne = une observation  
        5 2 ;  
        7 2 ;  
        9 1 ;  
        8 4 ]  
  
X_e = [ 1 1 ;  
        3 1 ;  
        2 2 ;  
        6 3 ;  
        3 3 ]  
  
c = 10  
nS = size(X_s, 1)  
nE = size(X_e, 1)
```

Réponse

```
# ----- MODELE LP -----  
  
using JuMP, GLPK  
modAD = Model(GLPK.Optimizer)  
  
@variable(modAD, w1)  
@variable(modAD, w2)  
@variable(modAD, d)  
  
@objective(modAD, Max, d)  
  
@constraint(modAD, [j=1:nS], X_s[j,1] * w1 + X_s[j,2] * w2 - d >= c)  
@constraint(modAD, [j=1:nE], X_e[j,1] * w1 + X_e[j,2] * w2 + d <= c)  
  
print(modAD)  
  
# ----- RESOLUTION -----  
  
optimize!(modAD)
```

Réponse

```
# ----- RESULTATS -----

println("Statut : ", termination_status(modAD))

w1_opt = value(w1)
w2_opt = value(w2)
d_opt  = value(d)

println("w1 = ", round(w1_opt, digits=2),
        "  w2 = ", round(w2_opt, digits=2),
        "  d = ", round(d_opt, digits=2))

# Calcul des scores
scores_succes = X_s * [w1_opt; w2_opt]
scores_echecs = X_e * [w1_opt; w2_opt]

println("Scores Succès : ", round.(scores_succes, digits=1))
println("Scores Échec  : ", round.(scores_echecs, digits=1))
```

Réponse

```
using Plots

# Calcul droite séparatrice et marges
x1r      = range(0, 12, length=200)
sep      = (c          .- w1_opt .* x1r) ./ w2_opt
marg_sup = (c + d_opt   .- w1_opt .* x1r) ./ w2_opt
marg_inf = (c - d_opt   .- w1_opt .* x1r) ./ w2_opt

# Affichage points
p = scatter(X_s[:,1], X_s[:,2],
            label="Succès", marker=:circle, ms=7, color=:darkgreen,
            title="Analyse discriminante par programmation linéaire",
            xlabel="x1", ylabel="x2")

scatter!(p, X_e[:,1], X_e[:,2],
        label="Échec", marker=:diamond, ms=7, color=:darkred)

# Affichage droites
plot!(p, x1r, sep,      lw=2, color=:black, label="Seuil c")
plot!(p, x1r, marg_sup, lw=1, ls=:dash, color=:gray, label="Marge +d")
plot!(p, x1r, marg_inf, lw=1, ls=:dash, color=:gray, label="Marge -d")

display(p)
```